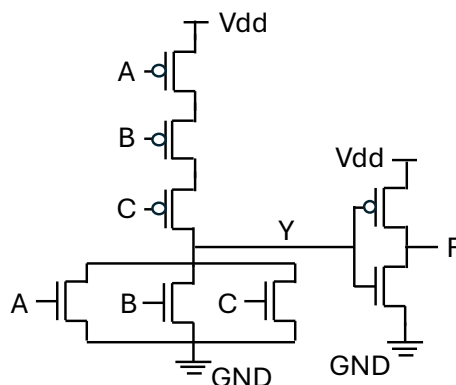


Homework Assignment 1

Due on 2025/09/28 23:59

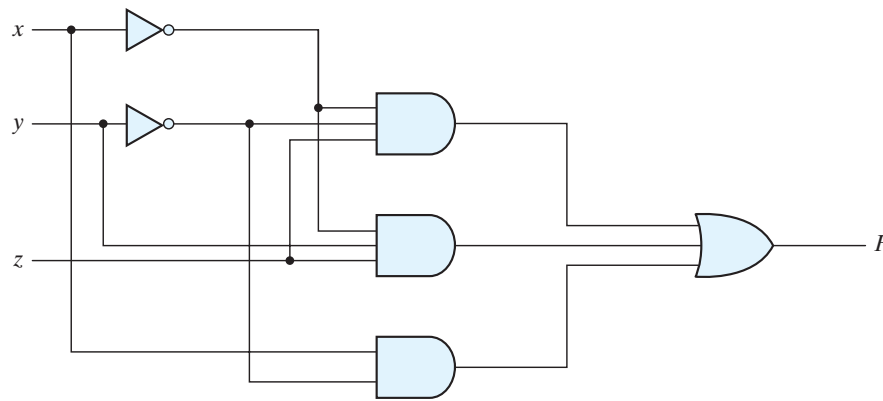
(Provide your intermediate results with the final answer to get the scores.)

1. (20%) The signed 2's complement representation of a number with a fractional part is the same as that of an integer except for the different position of the radix point.
For example, suppose $(32.1)_8$ is represented by an 11-bit signed 2's complement number with its higher 7 bits as the integer and lower 4 bits as the fractional part. Convert $(32.1)_8$ to its binary form, we have $(0\ 011\ 010.001\ 0)_2$. Note that the leftmost bit is the sign bit.
 - (a) (10%) Convert $(47.1875)_{10}$ to its signed 2's complement binary form as x . Consider the same 11-bit format in the example, i.e., with a 7-bit integer part and a 4-bit fractional part. What is x ? You must show how you derive the result step by step.
 - (b) (10%) Perform $z = (32.1)_8 - (47.1875)_{10}$ with the binary addition of 11-bit signed 2's complement numbers. You must show how you derive the result step by step. What is z in decimal? Does the calculation cause an overflow? You must give the reason.
2. (10%) The following design is a three-input logic gate.
 - (a) (8%) Derive the truth table of Y and F with the inputs A , B , and C . You must explain how you derive the result.
 - (b) (2%) What is this logic gate? Write down the Boolean function of F . Hint: You may search among the common logic gates in the lecture notes.



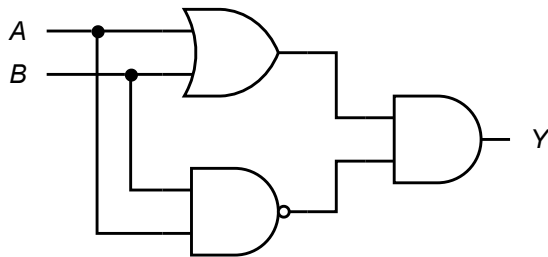
3. (10%) Prove that $(xy + x'y')' = (x' + y')(x + y)$ by using truth tables.
4. (10%) Draw the logic diagram corresponding to the following Boolean function directly, given the inputs x , y , and z (you don't need to simplify it). You must use **two-input gates** of AND, OR, INV, NAND, and NOR.

$$F = (xy)' \cdot (x + z) \cdot y'$$
5. (10%) Drive the Boolean functions directly from the following logic diagrams. You don't need to simplify the function.



6. (20%) For the following logic diagram:

(a) (10%) Drive the Boolean function Y . You don't need to simplify the function.



(b) (10%) Drive its truth table. Is it equivalent to any of the basic gates among INV, AND, NAND, OR, NOR, XOR, or XNOR? If yes, which one?

7. (20%) Perform the Verilog simulation of the 8-bit Binary Adder.

Hint:

Download **hw1_verilog.zip** which provides sample Verilog designs and their testbench:

majority_gate.v: gate-level design of the majority function

majority_func.v: functional design of the majority function

majority_t.v: testbench of the majority function using repeat statement to generate all possible input combinations

full_adder.v: functional design of the full adder

full_adder_t.v: testbench of the full adder with specific test patterns (four patterns in this example)

mystery.v: a mystery 3-input function **out**

binary_adder.v: functional design of the 8-bit binary adder

binary_adder_t.v: testbench of the 8-bit binary adder with four specific test patterns

Refer to the lecture notes to simulate the majority and full adder designs as exercises first.

(a) Visit EDA Playground website and create an account: <https://www.edaplayground.com>

(b) Reuse and revise the testbench majority_t.v to simulate mystery.v. Capture and provide screenshots of the final simulation result for both text log and waveform. Draw the truth table of the mystery function **out**.

Note: You might need to revise the testbench to reflect the module name change:

```
// instantiate the block
```

```
mystery m(out, count[0], count[1], count[2]);
```

(c) Copy-and-paste the binary adder design and testbench.

Add the fifth test pattern to perform $-7 + 15$. To do so, you simply insert additional statements in blue colors as follows:

```
...
// pattern 4
{a, b, ci} = {8'b00000110, 8'b11110011, 1'b0};
#10
$display("a = %b, b = %b, ci = %b | {co, si} = %b %b", a, b, ci, co, s);
// pattern 5
{a, b, ci} = {the test pattern you want to apply};
#10
$display("a = %b, b = %b, ci = %b | {co, si} = %b %b", a, b, ci, co, s);
$finish;
...
```

Verify the correctness of the simulation result.

- (d) Add the seventh pattern of your own to show the case when $co=1$. You need to list the inputs and output in binary and decimal and explain the arithmetic result.
- (e) Add the sixth pattern of your own to show an overflow condition. You need to list the inputs and output in binary and decimal and explain why there is an overflow.
- (f) Capture and provide screenshots of the final simulation results of (d) and (e) for both text log and waveform of binary values.